ThingIndex: Inteligência Documental com Proveniência Verificável sobre Modelo Relacional

Charles Eduardo Mello Guimarães[1]

[1] Instituto Federal do Paraná (IFPR) – Campus Palmas
Curso de Sistemas de Informação
Palmas – PR – Brasil

charles.eduardo.mello.guimaraes@hotmail.com

Resumo. *Quem consulta leis, normas e manuais precisa de respostas que possam ser conferidas no original. Por* Design Science Research*, construiu-se e avaliou-se o ThingIndex, que guarda o conteúdo de PDFs em tabelas relacionais, com cada parte do documento ligada a ele por chave estrangeira e à página de onde veio (nos parágrafos, também à região), busca os trechos por palavras (BM25) e mostra a fonte de cada resposta. Em três acervos públicos com perguntas escritas depois do desenvolvimento, a taxa de acerto foi de 69,5% com a pergunta original e de 88,7% com expansão da pergunta (diferença estatisticamente significativa,* $p < 0{,}001$*); na expansão, um LLM acrescenta termos técnicos à pergunta antes da busca, recurso experimental e desligado por padrão. No acervo de desenvolvimento, buscar trechos enviou ao modelo, na mediana, cerca de 1/26 dos* tokens *(unidades de texto) de entregar o documento inteiro. O erro mais grave passou pela conferência da fonte: o sistema citou, com a página certa, um artigo de uma lei já revogada, sem avisar que ele não vale mais.*

Palavras-chave: recuperação de informação, proveniência de dados, geração aumentada por recuperação, modelagem relacional, *Design Science Research*.

Abstract. *People who consult laws, regulations and manuals need answers they can check against the original. Following Design Science Research, this work built and evaluated ThingIndex, which stores the content of PDFs in relational tables, with each part of a document linked to it by a foreign key and to the page it came from (for paragraphs, also to the region), retrieves passages by keywords (BM25) and shows the source of each answer. On three public collections with questions written after development, accuracy was 69.5% with the original question and 88.7% with query expansion by an LLM, an experimental feature turned off by default (*$p < 0.001$*). On the development collection, retrieving passages used a median of 1/26 of the tokens of sending the whole document to the model. The most serious error passed the source check: the system cited, with the correct page, an article of a repealed law without warning that it no longer applies.*

Keywords: information retrieval, data provenance, retrieval-augmented generation, relational modeling, design science research.

1. Introdução

Muitas perguntas de trabalho têm resposta em um documento que já existe. Um técnico de segurança do trabalho precisa confirmar quem paga o equipamento de proteção; uma secretaria acadêmica precisa localizar a carga horária de um curso no projeto pedagógico; um servidor do setor de compras de uma prefeitura precisa saber se a modalidade "tomada de preços" ainda pode ser usada, caso que este trabalho mostra ser o mais difícil (Seção 8.6). A resposta está em leis, normas e regulamentos, guardados em pastas ou em sistemas que indexam o nome do arquivo, mas não o conteúdo. Achar a cláusula exige abrir o arquivo certo e lê-lo, e uma resposta errada vira retrabalho ou decisão mal fundamentada.

Os modelos de linguagem de grande escala (LLMs), que estão por trás dos assistentes de conversa, mudaram a expectativa: em vez de uma lista de arquivos, espera-se uma resposta pronta. A técnica mais usada para isso é a geração aumentada por recuperação (RAG): antes de responder, o sistema busca trechos relevantes nos documentos e os entrega ao modelo junto com a pergunta [Lewis et al. 2020]. Para quem responde por uma decisão, porém, a resposta só tem valor se puder ser conferida no original. Uma resposta correta sem a página de origem obriga o leitor a refazer a busca, e uma resposta errada com uma citação plausível tende a passar despercebida.

Neste trabalho, um sistema é *auditável* quando cada fonte que ele mostra é *verificável*: aponta o documento e a página de onde veio (nos parágrafos, também a região), e essa indicação vem do banco de dados, e não do texto escrito pelo modelo. Na literatura de bancos de dados, isso se chama proveniência *where* [Buneman et al. 2001]. Chama-se de *inteligência documental* a capacidade de um sistema organizar e recuperar o conteúdo dos documentos, e não só o nome e os metadados. A pergunta de pesquisa é: *é possível sustentar um sistema de inteligência documental com auditabilidade utilizando exclusivamente modelagem relacional, sem depender de bancos vetoriais ou grafos como camada primária de recuperação?*

A expressão "sustentar um sistema com auditabilidade" foi desdobrada em três condições que os dados podem confirmar ou não, retomadas na Discussão: (a) toda fonte mostrada ao usuário tem origem lida do banco; (b) a busca sobre as unidades guardadas em tabelas, com a nota calculada na aplicação e sem vetores, encontra o trecho certo em perguntas que não foram usadas no desenvolvimento; e (c) conferir a fonte permite confirmar que a resposta está certa. A condição (c) vai além da definição de fonte verificável e foi incluída porque é o que importa para quem usa a resposta.

Para responder, foi construído o ThingIndex. O sistema extrai a estrutura dos PDFs (páginas, parágrafos com posição, tabelas e sumário), guarda essa estrutura em tabelas do PostgreSQL, busca os trechos por palavras, com o algoritmo BM25, e devolve com cada fonte a indicação que permite abri-la no documento. O LLM só entra no final, para redigir a resposta a partir das fontes encontradas. A Figura 1 resume o caminho de uma pergunta. Não usar um banco de vetores, a escolha mais comum em RAG, foi uma decisão de projeto, tomada por simplicidade de operação; a Seção 8 mede parte desse custo, no *corpus* de controle (Seção 8.1).

As contribuições são:

1. o sistema, implementado e testado, em que cada unidade recuperável é ligada ao

Pergunta do usuário → **busca BM25** nas tabelas da coleção → até **9 fontes**, cada uma com documento e página lidos do banco → o **LLM** redige a resposta só a partir delas → a tela mostra a resposta e **abre cada fonte no PDF**, com o parágrafo destacado

Figura 1. Caminho de uma pergunta no ThingIndex. O modelo não escolhe as fontes: elas são escolhidas pela busca e lidas do banco antes de o modelo escrever.

documento por chave estrangeira e carrega a página e, nos parágrafos, a região, com a chave conferida por teste (Seção 6.3);
2. um protocolo de avaliação que deixa registrada qualquer mudança feita depois de medir: hipóteses registradas antes de medir, gabaritos travados por *hash*, a mesma pergunta escrita de dois jeitos (uma vez com as palavras do documento e outra com as palavras de quem não o leu) e registro das correções feitas depois de medir;
3. resultados em português, em sete acervos (seis de PDFs reais e um sintético, de controle), sobre acerto, origem das fontes e custo, incluindo um erro que a fonte verificável não detecta: a resposta que cita, com a página certa, uma norma revogada.

O texto segue esta ordem: objetivos (Seção 2), referencial teórico (Seção 3), trabalhos relacionados (Seção 4), metodologia (Seção 5), o sistema (Seção 6), o desenho da avaliação (Seção 7), os resultados (Seção 8), a discussão (Seção 9) e a conclusão (Seção 10). Diagramas, telas e tabelas completas estão nos anexos, no repositório do projeto (Apêndice A).

2. Objetivos

O objetivo geral é projetar, implementar e avaliar o ThingIndex, um sistema de inteligência documental que organiza e recupera o conteúdo de coleções de PDFs com fontes verificáveis, e comparar sua recuperação com alternativas densas e com a entrega do documento inteiro a um LLM.

Os objetivos específicos são:

1. modelar em tabelas relacionais as estruturas extraídas de PDFs (páginas, parágrafos, tabelas, sumário e trechos), com a coordenada de origem de cada unidade exigida pelo esquema;
2. implementar o sistema completo, com indexação assíncrona resiliente a falhas, recuperação por coleção e *chat* que exibe as fontes;
3. definir métricas de auditabilidade e um protocolo de avaliação com gabaritos independentes da saída do sistema;
4. comparar a recuperação lexical com a densa, exata e aproximada, em qualidade de ordenação num *corpus* de controle e em estabilidade;
5. comparar, em acerto, proveniência e custo, o contexto recuperado com a entrega do documento inteiro ao modelo;
6. medir o sistema em acervos de domínios distintos e testar intervenções dirigidas às causas de erro encontradas.

3. Referencial Teórico

Esta seção apresenta só os conceitos de que o restante do texto depende. A Tabela 1 resume os termos mais usados, para consulta durante a leitura.

Tabela 1. Termos usados no artigo, no sentido em que são usados aqui.

Termo	Significado
LLM	Modelo de linguagem de grande escala: lê um texto de entrada (o *prompt*) e escreve a resposta.
Token	Pedaço de palavra que o LLM processa. O custo de uma chamada cresce com os *tokens* enviados, e cada modelo aceita um máximo deles (a *janela*).
RAG	Geração aumentada por recuperação: buscar trechos nos documentos e entregá-los ao LLM junto com a pergunta.
Busca lexical	Busca por palavras: um trecho é bom se contém as palavras da pergunta. O BM25 é o algoritmo usado.
Busca densa	Busca por significado: pergunta e trecho viram listas de números (vetores), e trechos de sentido parecido ficam próximos.
Índice aproximado	Estrutura que acelera a busca densa olhando só parte dos vetores, ao custo de às vezes perder o mais próximo.
Unidade, fonte	Unidade é cada parte do PDF guardada numa linha de tabela (página, parágrafo, linha de tabela, seção, trecho). Fonte é a unidade mostrada ao usuário junto da resposta: uma seção, uma página ou uma linha de tabela.
Braço	Cada configuração comparada num experimento (por exemplo, BM25 contra busca densa).
Gabarito	Lista de perguntas, cada uma com a página onde está a resposta e o padrão que uma resposta certa precisa conter.
Literal, paráfrase e par	As duas redações de uma mesma pergunta: com as palavras do documento (literal) e com as palavras de quem não leu o documento (paráfrase). As duas juntas formam um *par*.
Ponto percentual	Diferença entre duas porcentagens: de 40% para 50%, o aumento é de 10 pontos percentuais.
Por resposta e por pergunta	Cada pergunta foi feita três vezes. “Por resposta” conta cada uma das três; “por pergunta” só conta acerto quando as três acertam.
Página na fonte e página no texto	Página na fonte: a página certa está entre as fontes que a tela mostra; vem do banco e não pode ser inventada. Página no texto: a página certa está escrita no texto da resposta; vem do modelo e pode estar errada.

3.1. Busca por palavras e busca por significado

A recuperação de informação estuda como encontrar, numa coleção, os itens que atendem a uma necessidade de informação [Manning et al. 2008]. A forma mais antiga é a busca por palavras, ou *lexical*. O BM25, usado no ThingIndex, dá uma nota a cada trecho a partir de três ideias [Robertson e Zaragoza 2009]: cada palavra da pergunta encontrada no trecho soma pontos; uma palavra rara na coleção vale mais que uma comum, porque diz mais sobre o assunto (num acervo de normas de segurança, "EPI" vale mais que "trabalho"); e repetir a palavra ajuda cada vez menos, enquanto trechos longos são penalizados, para que um trecho não vença só por ter mais palavras. Os parâmetros k_1 e b regulam essas duas últimas ideias. A primeira versão do ThingIndex só contava ocorrências, sem dar peso à raridade. O BM25 é simples e difícil de superar: num conjunto de testes em domínios variados, o BEIR, ele foi melhor que a maior parte dos métodos densos da época [Thakur et al. 2021].

A fraqueza da busca por palavras é o descasamento de vocabulário. Furnas et al. [1987] mostraram que duas pessoas raramente escolhem a mesma palavra para a mesma coisa: quem pergunta "quem paga o capacete e a luva?" não usa as palavras da norma, que fala em equipamento de proteção individual. Uma solução é a *expansão da pergunta*, que acrescenta à busca termos relacionados. Com LLMs, pode-se pedir ao próprio modelo esses termos, que é o método de Jagerman et al. [2023] e o usado neste trabalho.

A outra família é a busca *densa*, ou por significado. Um modelo treinado converte cada texto numa lista de centenas de números, chamada vetor, de modo que textos de sentido parecido fiquem com vetores próximos. A pergunta também vira vetor, e a busca devolve os trechos mais próximos dela. A vantagem é encontrar "carro" quando se pergunta "automóvel". Este trabalho usa o `multilingual-e5-base` [Wang et al. 2024], um modelo multilíngue que gera vetores de 768 números.

Comparar a pergunta com todos os vetores guardados é exato, mas fica lento com milhões deles. Por isso se usam *índices aproximados*: o IVF separa os vetores em grupos, como gavetas, e na consulta abre só as gavetas mais próximas; o HNSW liga cada vetor a seus vizinhos e caminha por essa rede até a região da pergunta. Os dois podem deixar de fora um vizinho que a busca exata acharia, e a documentação do pgvector, extensão que acrescenta esses índices ao PostgreSQL, avisa que o resultado pode diferir do da busca exata [Kane 2026]. Por fim, listas de resultados de métodos diferentes podem ser combinadas pela fusão por posição recíproca (RRF), que soma o inverso da posição de cada item em cada lista [Cormack et al. 2009].

3.2. Geração aumentada por recuperação

Um LLM responde a partir do que viu no treinamento e, sem uma fonte externa, produz com frequência afirmações fluentes e falsas, o que se chama de alucinação [Ji et al. 2023]. A RAG, proposta por Lewis et al. [2020], combina um buscador com o gerador: os trechos encontrados entram no *prompt* junto com a pergunta, e o modelo é instruído a responder a partir deles. Na forma mais comum, os documentos são cortados em blocos de tamanho fixo, convertidos em vetores e guardados num banco de vetores [Gao et al. 2023]; nesse corte, a ligação de cada bloco com a página do original fica a cargo de quem monta o sistema.

Alguns limites conhecidos dessa técnica aparecem nos experimentos deste trabalho. O modelo aproveita pior a informação que está no meio de um texto longo [Liu et al. 2024]. Quando as fontes entregues se contradizem, como uma lei e a lei que a revogou, o modelo não resolve o conflito de forma confiável [Xu et al. 2024]. E o modelo tende a concordar com a premissa da pergunta mesmo quando ela é falsa, comportamento chamado de bajulação [Sharma et al. 2024]. Com modelos que aceitam textos muito longos, uma alternativa à RAG é entregar o documento inteiro; Li et al. [2024] concluíram que o documento inteiro acerta mais quando há recursos para ele, e a recuperação custa muito menos.

3.3. Modelo relacional e proveniência

No modelo relacional de Codd [1970], os dados ficam em tabelas, e as ligações entre elas são garantidas por chaves: uma chave estrangeira faz o banco recusar uma linha que aponte para outra que não existe. Quem garante isso é o próprio banco, mesmo que o programa tenha algum erro.

Proveniência é a informação sobre a origem de um dado. Buneman et al. [2001] distinguem a proveniência *where*, que diz onde está o valor de origem, da *why*, que diz quais dados justificam um resultado. No ThingIndex, o interesse é a *where*: cada unidade recuperada aponta documento, página e região. É preciso separar isso de outra pergunta: a frase da resposta é sustentada pela fonte citada? Essa é a noção de *atribuição* [Rashkin et al. 2023]. Saber de onde a fonte veio é condição para conferir a atribuição, mas não a substitui: o sistema pode apontar a página certa de um trecho que não sustenta o que a resposta diz.

3.4. Integração assíncrona

Indexar um PDF leva tempo, então o envio só registra o pedido, e um processo separado faz o trabalho depois, avisado por uma fila de mensagens. O problema é que gravar no banco e publicar na fila são duas escritas sem transação comum: se o sistema cair entre elas, o documento fica registrado sem que ninguém seja avisado. O padrão *transactional outbox* resolve isso gravando a mensagem numa tabela, na mesma transação do registro, e publicando-a depois [Richardson 2018].

4. Trabalhos Relacionados

A Tabela 2 compara o ThingIndex com os trabalhos mais próximos quanto a duas escolhas: como as fontes são escolhidas e de onde vem a indicação de origem que o usuário vê.

Bibliotecas como o LangChain servem para montar aplicações com LLMs e deixam a origem das fontes a cargo de quem as usa; o ThingIndex é uma aplicação completa que fixa uma escolha (unidades da estrutura do PDF, com página) e a avalia. O GraphRAG mira perguntas que exigem visão de conjunto do acervo, mas depende do LLM já na indexação, e o que recupera pode ser um resumo gerado. O PageIndex é o mais próximo: também usa a estrutura do PDF e dispensa vetores, mas é o modelo que escolhe a seção. Assistentes comerciais que aceitam PDFs escalam e atendem várias pessoas, o que o ThingIndex não faz; ele difere deles por poder rodar inteiramente na instalação da organização, com um modelo aberto, e por ter a garantia de origem visível no esquema e no código. Não houve comparação experimental com esses produtos nem com o PageIndex.

Tabela 2. Comparação com trabalhos relacionados.

Trabalho	Como escolhe as fontes	Origem da fonte
LangChain [Chase 2022] e bibliotecas semelhantes	O desenvolvedor escolhe o corte em blocos e o buscador	Os metadados que o desenvolvedor anexar ao bloco
GraphRAG [Edge et al. 2024]	Grafo de entidades e resumos construídos por LLM	Resumos gerados, e não o texto original
PageIndex [VectifyAI 2025]	O LLM navega a árvore de seções do PDF	A seção escolhida pelo modelo; sem avaliação publicada
Assistentes comerciais	Fechado	Referência ao documento, sem garantia que se possa inspecionar
ThingIndex	BM25 sobre unidades guardadas em tabelas. A escolha das fontes é do BM25; o LLM pode gerar resumos que entram no índice e, com a expansão ligada, sugerir palavras de busca	Documento ligado por chave estrangeira; página e região gravadas na indexação e lidas do banco

Quanto à avaliação, ferramentas como o RAGAS [Es et al. 2024] usam o próprio LLM como juiz das respostas. Este trabalho preferiu gabaritos escritos à mão, com página-alvo e padrão de resposta, para que o juiz não seja outro modelo. Os trabalhos acima ou propõem ferramentas sem avaliar a origem das fontes, ou avaliam respostas sem tratar de como a origem é guardada. Este trabalho junta as duas coisas num sistema completo, medido em documentos reais em português.

5. Metodologia

O trabalho é uma pesquisa aplicada conduzida segundo a *Design Science Research* (DSR), em que se constrói um artefato para resolver um problema relevante e se avalia o artefato com rigor [Hevner et al. 2004]. Seguiu-se o processo de seis atividades de Peffers et al. [2007]:

1. **Identificação do problema**: consulta a acervos normativos sem fonte verificável (Seção 1). Não houve levantamento com usuários; o problema foi caracterizado por casos de uso típicos e pela literatura.
2. **Objetivos da solução**: recuperar trechos com origem garantida pelo esquema, com custo por pergunta que não cresça com o tamanho do documento (Seção 2).
3. **Projeto e desenvolvimento**: o ThingIndex (Seção 6).
4. **Demonstração**: uso sobre seis acervos de PDFs reais e um *corpus* sintético de controle (Seção 7).
5. **Avaliação**: experimentos de busca, de origem das fontes e de geração da resposta (Seção 8). A utilidade para usuários finais não foi avaliada.
6. **Comunicação**: este artigo e o repositório do projeto, com código, dados, *scripts* de análise e anexos.

O desenvolvimento foi feito em ciclos: cada experimento que revelava uma limitação gerava uma mudança no sistema, medida no ciclo seguinte. Esses ciclos usaram um único acervo, o de ensino, que por isso funcionou como *conjunto de desenvolvimento*: o sistema foi ajustado olhando para as suas 72 perguntas, e seus resultados tendem a ser otimistas. Os gabaritos dos demais acervos foram escritos depois, sobre documentos ainda não usados, e funcionam como *conjunto de teste*. Para evitar que os ajustes sucessivos inflassem os números, foram adotadas as práticas a seguir:

- **Pré-registro**: nas duas intervenções testadas (Seções 8.8 e 8.9), a hipótese, o critério de sucesso e a análise foram escritos e versionados antes do código.
- **Gabaritos travados**: cada gabarito registra o *hash* SHA-256 do seu conteúdo, e o programa de avaliação recusa um gabarito alterado sem atualizar o *hash*, o que deixa toda alteração visível no histórico.
- **Números gerados por *script***: os resultados ficam em arquivos CSV, uma linha por pergunta e repetição, e as tabelas do artigo são geradas ou conferidas a partir deles.
- **Registro das correções feitas depois de medir**: toda alteração de gabarito ou de critério feita depois de uma medição foi registrada com data, motivo e efeito nos números (Anexo A8). As que mudam números deste artigo são citadas onde o número aparece.

6. O ThingIndex

6.1. Requisitos

São doze requisitos funcionais, cada um ligado a rotas da API cobertas por testes automatizados: cadastro e autenticação; organização em categorias e coleções; envio de documentos PDF; indexação assíncrona; enriquecimento opcional por LLM; recuperação de contexto por API; *chat* com as fontes citadas; abertura de cada fonte na página e na região de origem; configuração de provedores de LLM e de armazenamento com credencial cifrada; acompanhamento de *tokens* e do custo estimado; trilha de auditoria; e reprocessamento de falhas. A lista completa está no Anexo A1, e o diagrama de casos de uso, no Anexo A2.

Os principais requisitos não funcionais são: integridade referencial e transações no armazenamento da estrutura extraída; separação em camadas; segurança (senha com bcrypt, sessão por *token* JWT, credencial de provedor cifrada com AES-GCM e limite de requisições nas rotas de autenticação, envio e *chat*); execução reproduzível por Docker Compose; e recuperação determinística, isto é, a mesma pergunta sobre o mesmo índice traz as mesmas fontes na mesma ordem. Quanto à privacidade, com um modelo rodando na própria instalação nenhum conteúdo sai dela; com um provedor externo, como o usado nos experimentos, a pergunta e os trechos recuperados são enviados a ele, e cabe à organização a base legal para isso, nos termos da Lei Geral de Proteção de Dados (LGPD). Nesta versão o sistema é de uso individual, sem compartilhamento de coleções.

6.2. Arquitetura

O sistema tem quatro partes: o *frontend*, em Next.js e React; a API, em Go; o *indexer*, também em Go, que processa os documentos; e a infraestrutura, com PostgreSQL 16, a fila RabbitMQ e um armazenamento de arquivos compatível com S3 (MinIO no ambiente local). A API é organizada em camadas: os *handlers* HTTP validam a entrada, os casos de

uso contêm a regra de negócio e os repositórios isolam o acesso ao banco. Os provedores de LLM e de armazenamento ficam atrás de interfaces e podem ser trocados sem mudar os casos de uso; os experimentos usaram o Ollama.

No envio de um documento, a API grava o arquivo e, numa única transação, registra o documento, o pedido de indexação na tabela de *outbox* e o evento de auditoria. Um processo despachante lê os pedidos pendentes e os publica no RabbitMQ, marcando-os como publicados só depois que a fila confirma o recebimento. Se algo falhar no meio, o pedido é publicado de novo, e o *indexer* ignora o pedido repetido de um documento já indexado. Um documento parado na fila por mais de 30 minutos ganha um novo pedido, e um documento que falha em todas as tentativas é marcado como falho, com a causa registrada. O diagrama de sequência desse fluxo está no Anexo A3.

6.3. Modelo de dados e proveniência

O esquema tem 25 tabelas (Anexo A9). Quatro formam o núcleo (usuários, categorias, coleções e documentos). Seis guardam a estrutura extraída, uma por tipo de unidade: páginas, parágrafos, tabelas, nós do sumário, trechos e metadados. As demais servem à operação: *outbox*, tentativas de processamento, auditoria, provedores, conversas e *cache*. O recorte com o caminho da proveniência está no Anexo A4.

Cada linha de estrutura carrega sua origem: o documento, o número da página, a posição na ordem de leitura e, nos parágrafos, a região (a caixa delimitadora) que o parágrafo ocupa na página, extraída do PDF junto com o texto. O vínculo com o documento é uma chave estrangeira: o banco recusa um parágrafo que aponte para um documento inexistente. A página e a região, por outro lado, são gravadas pelo *indexer* na mesma linha, a página numa coluna e a região num campo JSON, e o banco não confere se a página existe no documento. Um trecho recebe a página do seu primeiro parágrafo, de modo que um trecho que atravessa duas páginas aponta só a primeira. A página mostrada ao usuário é a gravada na indexação e lida do banco, e o modelo não tem como alterá-la.

Ao revisar o código para este artigo, descobriu-se que essa chave estrangeira não existia: a biblioteca de mapeamento objeto-relacional usada, o GORM, só cria a restrição quando o modelo declara a associação, e os seis modelos de estrutura não a declaravam. A coluna existia e era obrigatória, mas o banco não conferia nada. A correção declarou a associação, removeu linhas órfãs na migração e acrescentou dois testes de integração: um confere no catálogo do PostgreSQL que as seis restrições existem, e o outro que o banco recusa um parágrafo sem documento; só a partir daí o banco passou a conferir o vínculo.

6.4. Indexação

O *indexer* extrai o texto com a ferramenta `pdftotext`, que entrega cada página separada e a posição de cada bloco e linha. A partir disso, regras de posição e de fonte montam os parágrafos na ordem de leitura, detectam tabelas pelo alinhamento das colunas, reconstroem a hierarquia de seções pelos títulos e pela numeração, e agrupam os parágrafos em trechos por seção. Uma tabela que atravessa páginas vira uma parte por página, para que cada célula tenha página. As regras são determinísticas e explicáveis, ao custo de errar em diagramações incomuns. Páginas escaneadas passam por reconhecimento óptico de caracteres (OCR); nesse caso não há região, e o sistema registra a ausência em vez de inventar uma.

Quando há um provedor de LLM configurado, o *indexer* também gera um resumo de cada página e de cada seção. Esse *enriquecimento* vem ligado por padrão, pode ser desligado nas preferências e fica em *cache* pelo conteúdo, para não ser pago de novo ao reindexar. Na configuração padrão, o resumo da página entra na busca junto com o texto da página, como descrito a seguir; a Seção 8.8 mede o efeito disso.

6.5. Recuperação e chat

Quando o usuário pergunta algo sobre uma coleção, a recuperação acontece em três passos (Anexo A5). Primeiro, o sistema carrega as unidades da coleção sempre com o mesmo número de consultas ao banco (nove, sem relação com as até nove fontes do terceiro passo), qualquer que seja o número de documentos; um teste de integração conta as consultas para 1 e para 20 documentos e falha se o número mudar. Cada consulta tem um limite (5.000 páginas e 20.000 parágrafos por coleção, por exemplo) e lê as linhas na ordem dos documentos, do primeiro enviado ao último, e, dentro de cada um, na ordem das páginas ou da leitura; acima do limite, as unidades excedentes, dos documentos enviados por último, ficam fora da busca sem aviso ao usuário. O evento só é registrado em *log*, o que é uma limitação desta versão. Segundo, as unidades recebem nota por BM25 ($k_1 = 1{,}2$; $b = 0{,}75$), com acentos e maiúsculas ignorados e singular e plural tratados como a mesma palavra. Na configuração padrão, usada em todos os experimentos, a nota da página considera o texto da página e o resumo gerado na indexação. Páginas, seções, trechos e linhas de tabela recebem nota em separado, e as notas são combinadas com pesos fixos por tipo de unidade, escolhidos durante o desenvolvimento (Apêndice B). Terceiro, o sistema seleciona até três linhas de tabela e até seis fontes entre seções e páginas, isto é, até nove fontes por resposta. Empates são desfeitos pelo identificador da unidade, de modo que a ordem é sempre a mesma para o mesmo índice.

O banco guarda as unidades e a origem de cada uma; a nota BM25 é calculada pela API, a cada pergunta, sobre as unidades carregadas, sem um índice de busca guardado à parte. A nota dos trechos não gera fonte: ela só ajuda a escolher o documento. Isso evita que um índice fique desatualizado em relação às tabelas, e o custo, que cresce com o tamanho da coleção, é aceitável nas coleções medidas, de até 10 documentos. Para acervos maiores, o índice de texto do próprio PostgreSQL fica como trabalho futuro.

As fontes selecionadas são enviadas à tela antes da resposta e, em seguida, entram no *prompt* do modelo, com a instrução de responder apenas a partir delas e de dizer quando a informação não consta. De cada fonte, vai ao modelo a linha de tabela ou o texto da página, cortado em 4.000 caracteres, até um total de 24.000 caracteres; as fontes que passam desse total entram só com um trecho de 220 caracteres. A expansão da pergunta (Seção 8.9) existe no código, mas fica desligada por padrão.

6.6. Interface

A tela principal é o *workspace* de uma coleção (Figura 2). Os documentos aparecem como páginas de PDF, e o *chat* fica na lateral. As fontes aparecem numeradas abaixo da resposta, com documento e páginas; ao selecionar uma delas, a tela vai até a página e destaca a região do parágrafo sobre o PDF, e o usuário confere a fonte sem sair da conversa. A ligação entre cada frase da resposta e a fonte que a sustenta não é exibida: o modelo cita a página no texto, e cabe ao leitor associá-la à fonte. As demais telas estão no Anexo A6.

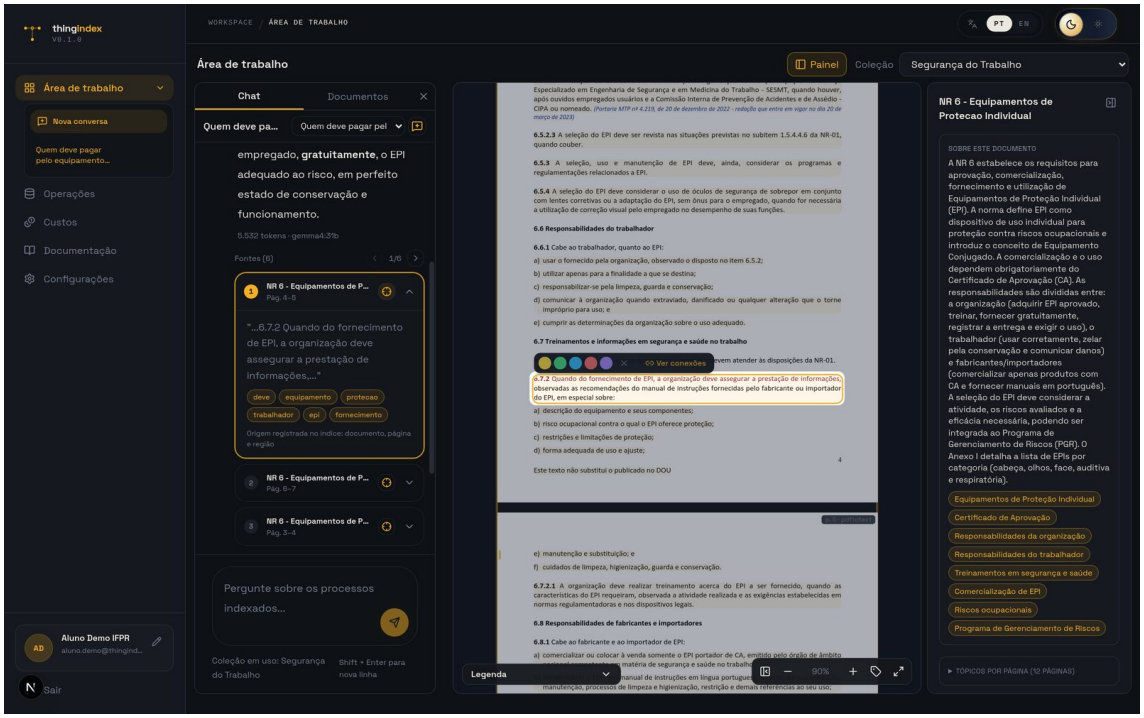


Figura 2. *Workspace* de uma coleção de normas de segurança do trabalho: *chat* com a primeira fonte expandida (à esquerda), PDF na página citada com o parágrafo destacado (ao centro) e o resumo do documento gerado na indexação (à direita).

6.7. Verificação do software

O *backend* tem 159 arquivos de teste, com 732 funções de teste, dos quais 26 são testes de integração, quase todos contra PostgreSQL, RabbitMQ ou MinIO reais; a cobertura medida foi de 56,0% das instruções. O *frontend* tem 85 arquivos de teste unitário e 16 arquivos, com 33 casos, de teste de interface em navegador. Há ainda um teste de caos, que derruba, um de cada vez e com o sistema em funcionamento, o RabbitMQ durante um envio, o PostgreSQL e o armazenamento de arquivos, e confere que o sistema volta a funcionar sem ser reiniciado; num cenário à parte, vinte envios simultâneos de PDF para a mesma coleção terminam indexados. As versões finais de todos os testes, inclusive o de caos (16 de 16 verificações), foram executadas localmente.

7. Desenho da Avaliação

A avaliação responde, nesta ordem, às seguintes perguntas: a busca por palavras é competitiva com a busca densa (Seções 8.1 a 8.3)? As fontes entregues têm origem verificável (Seção 8.4)? E o sistema completo, com o modelo redigindo a resposta, acerta, cita e custa quanto (Seções 8.5 a 8.10)? A Tabela 3 resume os acervos usados; C1 é o nome dado ao *corpus* de controle, o único acervo montado só para o teste. A *bateria* reúne os três acervos com perguntas escritas depois do desenvolvimento e medidos juntos, com três repetições: segurança do trabalho, produto e consumidor e licitações. O Anexo A10 traz, para cada acervo, os PDFs indexados, exceto o do edital municipal, que não está versionado, e um arquivo com todas as perguntas, a resposta esperada e a página onde ela está. A Tabela 4, no início da Seção 8, dá a resposta curta a cada uma.

Tabela 3. Acervos e gabaritos usados na avaliação. Repetições: quantas vezes cada pergunta foi feita.

Acervo	Documentos	Perguntas	Repetições	Seção
C1, recuperação	5 textos fictícios elaborados para o teste, de 111 a 166 palavras (jurídico, licitação, contratual, normativo e operacional)	18	–	8.1
Ensino	10 PDFs: 8 do IFPR e 2 normas externas (Lei 11.788/2008 e Resolução CNE/CES 5/2016)	72	3	8.5
Legislação	Leis 8.666/1993 e 14.133/2021	19	1	8.6
Segurança do trabalho	5 PDFs de 4 Normas Regulamentadoras (NR-1 em duas redações)	23	3	8.7
Produto e consumidor	Código de Defesa do Consumidor, manual de monitor, termo de garantia	23	3	8.7
Licitações	Leis 8.666, 14.133 e 10.520	25	3	8.7
Edital municipal	1 edital de concurso, 52 páginas	18	1	8.10

Gabaritos. Todas as perguntas foram escritas a partir dos PDFs, sem consultar o sistema. Cada uma declara o documento e a página onde está a resposta e um padrão de texto (expressão regular) que a resposta certa precisa conter; quando necessário, declara também padrões que invalidam a resposta, como os de uma recusa ("não há informação sobre isso"). Um teste automatizado confere que uma recusa genérica não conta como acerto em nenhuma pergunta que tenha resposta. Na bateria e no edital, a maior parte das perguntas foi escrita duas vezes: uma versão *literal*, com as palavras do documento, e uma *paráfrase*, com as palavras de quem não leu o documento, como "Quem deve fornecer o EPI ao trabalhador e em que condições?" e "Quem paga o capacete e a luva que eu uso no serviço?". As duas versões da mesma pergunta formam um *par*. Na bateria, as 9 perguntas negativas não têm par. Os pares existem porque quem escreve a pergunta logo depois de ler a página tende a copiar as palavras dela, e isso favorece a busca por palavras; comparar as duas versões mostra o tamanho desse favorecimento.

Acerto e origem. Uma resposta é *correta* quando satisfaz o gabarito; nas perguntas *negativas*, que não têm resposta no acervo, quando declara que a informação não consta. Salvo indicação, a taxa de acerto é *por resposta*, e os testes estatísticos usam o acerto *por pergunta* (Tabela 1), porque as três repetições de uma pergunta não são independentes e o teste de McNemar precisa de um único resultado, certo ou errado, por pergunta. A origem é medida de dois modos: a *página na fonte*, que vem do banco e só existe quando há busca, e a *página no texto*, escrita pelo modelo, que vale para qualquer braço e por isso permite comparar a busca com a entrega do documento inteiro. Nenhuma das duas confere se o trecho sustenta a frase da resposta.

Métrica de busca. Em C1, cada consulta tem o texto certo conhecido, e cada método devolve uma lista ordenada. O artigo usa a medida mais simples: em quantas das 16 consultas com resposta o primeiro resultado da lista era o texto certo (na literatura, P@1). As demais medidas da área estão no Anexo A7.

Origem no índice. Mede-se a porcentagem de unidades indexadas com *origem completa* no banco, isto é, com documento, página, posição e conteúdo preenchidos. Confere-se também se o texto de cada parágrafo extraído aparece, letra por letra (ignorando maiúsculas e espaços), no documento de onde o sistema diz que ele veio.

Custo e modelo. O custo é medido em *tokens* enviados ao modelo por pergunta. Não se usou valor em reais porque o preço muda com o provedor e com o tempo. Todos os experimentos de geração usaram o mesmo modelo aberto, `gemma4:31b`, servido pelo Ollama em nuvem, isto é, por um provedor externo; o uso com um modelo rodando na própria instalação é possível, mas não foi medido. Nem a versão exata do modelo nem a sua *temperatura* (o grau de variação que o modelo se permite ao escrever) foram fixadas; as três repetições medem a variação que resta.

Estatística. Os resultados são dados em contagem ("13 de 16") ou, na bateria, em porcentagem das respostas, e toda diferença de acerto que o artigo trata como real vem com o valor-p: a chance de aparecer uma diferença pelo menos desse tamanho se as duas condições fossem, na verdade, iguais; abaixo de 0,05 (5%), a diferença é tratada como real. Para comparar duas condições nas mesmas perguntas usou-se o teste de McNemar exato [McNemar 1947], escolhido porque cada pergunta dá certo ou errado nas duas condições, e só as perguntas em que elas discordam dizem qual é melhor. As duas comparações principais com esse teste, a redação da pergunta e a expansão, deram $p < 0{,}001$, valor que continua abaixo de 0,05 mesmo corrigido pelo número de comparações. O intervalo de confiança de 95% é a faixa em que o resultado provavelmente ficaria com outras perguntas do mesmo tipo: quanto menos perguntas, mais larga a faixa. Foi calculado sorteando 10.000 vezes, com reposição, novos conjuntos das mesmas perguntas (*bootstrap*), com as três respostas de cada pergunta sorteadas juntas, o que dispensa supor uma distribuição para os dados. Em C1 usou-se o teste de aleatorização pareado, que troca ao acaso, consulta a consulta, os resultados dos dois métodos e verifica com que frequência surge uma diferença tão grande quanto a real; ele foi preciso porque nem todas as 5 medidas do Anexo A7 são certo ou errado, e, numa medida de certo ou errado como o primeiro resultado, dá o mesmo valor-p que o McNemar exato. Como ali foram feitas 90 comparações (75 entre os textos inteiros e 15 entre as seções), aplicou-se, em cada grupo, a correção de Holm, que exige valores-p menores quanto mais comparações se fazem, para que nenhuma pareça significativa só por sorte.

8. Resultados

A Tabela 4 resume o que foi medido; as subseções trazem o detalhe.

8.1. Busca por palavras contra busca densa

C1 é um *corpus* de controle: cinco textos curtos e fictícios, elaborados para o teste, com 18 consultas, das quais 16 têm resposta e duas não. As consultas são de três tipos: literais (palavras tiradas do documento), paráfrases (a mesma necessidade com outras palavras) e ruidosas (com erros de digitação e sem acentos). Foram comparados seis métodos nas

Tabela 4. Resumo dos resultados. As porcentagens são da bateria e contam respostas (cada pergunta foi feita três vezes); "x de y" conta consultas, unidades, perguntas ou pares.

O que se perguntou	Resultado	Seção
Busca por palavras ou por significado: qual acha mais?	Sem diferença detectável: 13 de 16 contra 11 de 16 consultas no primeiro resultado	8.1
Cada unidade tem a origem gravada no banco?	Sim: 155 de 155 unidades de C1; parte veio de ajustar a métrica	8.4
Buscar trechos custa menos que mandar o PDF inteiro?	Sim, no acervo de desenvolvimento: na mediana, cerca de 1/26 dos *tokens*, com 72 de 72 perguntas certas contra 71 de 72	8.5
Quanto o sistema acerta em perguntas novas?	69,5% das respostas, cerca de 7 em 10 (intervalo de 95%: 59,2% a 79,3%)	8.7
A redação da pergunta muda o acerto?	Sim: 88,2% com as palavras do documento e 41,9% com outras; com o gabarito revisado, 14 pares a 0 ($p < 0{,}001$); com o original, 8 a 2 ($p = 0{,}109$)	8.7
Expandir a pergunta corrige isso?	Em parte: de 69,5% para 88,7% no total, e as paráfrases chegam a 83,9%; 14 perguntas ganhas e nenhuma perdida ($p < 0{,}001$); experimental, desligada por padrão	8.9
Fonte certa garante resposta certa?	Não: o artigo de uma lei revogada foi citado com a página certa	8.6

mesmas consultas: a contagem de palavras, que era o pontuador da primeira versão do sistema; o BM25; a busca densa exata; a mesma busca densa com um índice aproximado; e duas combinações, a da contagem com o BM25 e a do BM25 com a busca densa. Mediu-se em dois níveis: escolhendo entre os 5 textos inteiros e escolhendo entre 19 seções. A Tabela 5 traz os resultados. A versão aproximada da busca densa acertou o primeiro resultado tantas vezes quanto a exata (11 de 16).

Tabela 5. Busca em C1: em quantas das 16 consultas com resposta o primeiro resultado era o texto certo. Entre as 19 seções, só as buscas por palavras foram medidas. Os seis métodos e as demais medidas estão no Anexo A7.

	Primeiro resultado certo	
Método	Entre 5 textos	Entre 19 seções
Contagem de palavras (versão anterior)	12 de 16	9 de 16
BM25 (usado no sistema)	13 de 16	12 de 16
Busca densa	11 de 16	–
BM25 + busca densa (híbrida)	13 de 16	–

Entre os 5 textos, o BM25 acertou o primeiro resultado em 13 das 16 consultas (81,2%) e a busca densa em 11 (68,8%): duas consultas de diferença, numa tarefa pouco

exigente, em que até um sorteio acertaria 1 vez em 5. Escolher entre 19 seções é mais difícil e mais próximo do que o sistema faz; ali, o BM25 acertou 12 (75,0%) e a contagem, 9 (56,2%), e por isso a contagem foi trocada pelo BM25 em todos os experimentos seguintes. Os dois tipos de busca erram em consultas diferentes: as buscas por palavras acertaram as 5 consultas literais nos dois níveis e erraram mais nas paráfrases, e a densa errou 2 das 5 literais (Anexo A7, Tabela 2). No primeiro resultado, juntar as duas (busca híbrida) não superou o BM25 sozinho.

Nenhuma diferença entre métodos foi estatisticamente significativa, nem antes da correção de Holm: o menor valor-p foi 0,125 entre os textos e 0,062 entre as seções. Em nenhuma comparação dois métodos discordaram em mais de 8 das 16 consultas, e com tão poucas discordâncias só uma diferença muito grande seria detectada. Neste *corpus*, portanto, a busca por palavras e a densa ficaram próximas.

8.2. Quando não há resposta

Nas duas consultas de C1 sem resposta, todos os braços devolveram resultados: na busca por palavras, qualquer palavra em comum já gera nota positiva, e na densa as notas de textos relevantes e irrelevantes quase se sobrepõem. A nota da busca não serve para decidir que não há resposta. O sistema completo, porém, se comporta de outro modo: instruído a responder só a partir das fontes, o modelo declarou que a informação não consta em todas as perguntas negativas medidas, as 5 do acervo de ensino e as 9 da bateria, nas três repetições. Portanto, o que identifica a falta de resposta é a instrução dada ao modelo, já que a busca sempre devolve alguma fonte.

8.3. Estabilidade dos resultados

Um requisito do sistema é que a mesma pergunta traga sempre as mesmas fontes. Cada método de C1 foi executado 30 vezes, e as listas completas foram comparadas. Contagem, BM25, a fusão dos dois e a busca densa exata deram listas idênticas em todas as execuções. No índice aproximado, configurado com muitas gavetas e uma só aberta por consulta, mudar o sorteio que forma as gavetas ou a ordem de inserção dos documentos produziu até quatro listas diferentes para a mesma consulta.

Essa configuração, porém, não é a que o pgvector recomenda. Medido no próprio pgvector, com 1.000 a 20.000 vetores sintéticos (Anexo A7, Tabela 5), o índice IVF com a configuração recomendada deu sempre o mesmo resultado da busca exata; a configuração usada no teste mudou todas as 20 consultas, e o HNSW com os parâmetros padrão foi exato com 1.000 vetores, mas mudou algumas consultas a partir de 5.000. Nas coleções que o ThingIndex atende, de até alguns milhares de unidades, um índice de vetores bem configurado também seria estável: a estabilidade não é uma vantagem exclusiva da busca por palavras.

8.4. Origem das fontes no índice

C1 é texto sem paginação, e nele toda unidade fica na página 1. A porcentagem de origem completa mostra que cada unidade tem a origem gravada, mas não mostra que a página gravada esteja certa; a página certa é conferida por testes unitários com parágrafos em páginas diferentes e, nos PDFs reais, pela página na fonte nas seções seguintes. A região do parágrafo não foi avaliada em separado.

Na versão atual, as 155 unidades indexadas de C1 têm a origem completa. A primeira medição deu 51 de 155: as outras 104 não tinham página. Dezenove eram um defeito de modelagem: a tabela de trechos não tinha coluna de página, e corrigir o esquema levou a 70 de 155. As outras 85 não eram defeito: eram estatísticas do documento inteiro, como o número de parágrafos, que não estão em página nenhuma, e passaram a declarar que sua origem é o documento inteiro. Parte do 155 de 155 final, portanto, vem de ajustar a métrica, e não de corrigir o sistema (Anexo A7). Os 20 parágrafos extraídos de C1 aparecem todos, letra por letra, no documento de origem. Essa conferência avalia a extração e não verifica as páginas que o modelo escreve nas respostas.

8.5. Trechos recuperados contra documento inteiro

Este experimento mede o sistema como o usuário o encontra e o compara com a alternativa de entregar o documento inteiro ao modelo. No braço **T** (de *trechos*), o avaliador usa a mesma rota de *chat* que a tela usa. No braço **F** (de *full*, documento inteiro), o mesmo modelo recebe, sem nenhuma busca, o texto completo do documento que contém a resposta, com o número de cada página marcado; nas perguntas negativas, recebe cinco documentos. F leva vantagem, porque já recebe o documento certo, algo que um usuário não saberia de antemão; por isso, mostra o melhor resultado que a entrega do documento inteiro poderia ter. O acervo de ensino tem 10 PDFs e 72 perguntas, de quatro tipos: prosa, tabela, calendário e negativa.

Como este é o acervo de desenvolvimento, o resultado é otimista. O braço T foi ajustado várias vezes com base nessas 72 perguntas, e o gabarito passou por quatro versões no mesmo período. No começo, numa única rodada, T acertava 67 das 72 perguntas; ao final, acertou as 72 nas três repetições (216 respostas). O braço F foi executado uma vez, sem ajuste. Doze perguntas tinham sido usadas diretamente para calibrar o sistema, e os resultados sem elas são dados à parte.

Acerto. T acertou as 72 perguntas nas três repetições e F, 71: F errou a mesma negativa nas três. Sem as 12 perguntas de calibração, o resultado é o mesmo (60 de 60 e 59 de 60). A diferença é de uma pergunta, uma única discordância que nenhum teste distinguiria do acaso, e no acervo em que T foi ajustado: buscar antes de perguntar não custou acerto aqui.

Custo. Comparando pergunta a pergunta, F enviou ao modelo, na mediana, 26,5 vezes mais *tokens* que T; é daí que vem o "cerca de 1/26" do resumo. Usou-se a mediana porque poucas perguntas sobre documentos longos puxariam a média para cima. Em 22 das 72 perguntas, todas sobre documentos curtos, T gastou mais que F, porque várias páginas recuperadas somam mais que o documento inteiro. O custo de T depende do número de fontes, e o de F cresce com o documento: numa execução exploratória com outro modelo, de janela de 131 mil *tokens*, o provedor recusou, no braço F, o *prompt* das perguntas negativas, que leva cinco documentos (227.649 *tokens* na primeira delas). As médias e a faixa por pergunta estão no repositório (Apêndice A).

Origem. A página no texto esteve correta em 189 de 201 respostas em T e em 189 de 201 em F, aceitando também o número impresso na própria página, que em alguns documentos difere da posição no arquivo; só pela posição no arquivo, F cairia para 173. O que muda é de onde vem a página. Em F, ela só existe no texto escrito pelo modelo, e na pergunta que F errou a resposta afirma um dado inexistente e o atribui a uma página do

documento. Em T, a página na fonte vem do banco e não pode ser inventada pelo modelo; a página certa esteve entre as fontes em 195 de 201 respostas. O modelo, porém, também escreve páginas no texto em T, e essas podem estar erradas.

Com acerto de 100,0%, este acervo deixou de distinguir boas e más versões do sistema, em parte porque o sistema foi ajustado nele. Os acervos seguintes têm perguntas escritas depois do desenvolvimento, e são eles que valem como resultado.

8.6. Legislação com norma revogada

O acervo de legislação tem duas leis: a 8.666/1993 e a 14.133/2021, que a revogou. Conforme a data e o contrato, a mesma pergunta pode ter respostas diferentes, e os dois textos usam quase o mesmo vocabulário, o que é difícil para qualquer busca por semelhança. Xu et al. [2024] mostram que, diante de fontes que se contradizem, os modelos não resolvem o conflito de forma confiável. Das 19 perguntas, as *divergentes* exigem distinguir as duas leis, e responder só pela revogada conta como erro.

O sistema acertou 15 das 19 perguntas; nas 5 divergentes, acertou 3. Uma das quatro respostas erradas foi recusada por um erro do gabarito, que exigia "10 (dez) anos" quando a resposta dizia "10 anos"; o gabarito não foi alterado depois de medir, e o número oficial continua 15 de 19. A falha que motivou este acervo apareceu nas divergentes: perguntado se a modalidade "tomada de preços" existe, o sistema respondeu que sim e citou corretamente o art. 22 da Lei 8.666, sem dizer que ele foi revogado. Quem conferir a fonte encontra o artigo citado na página indicada, sem nada que avise que ele já não vale. O mesmo aconteceu com o limite de dispensa de licitação para obras. Com uma repetição e 19 perguntas, o resultado é descritivo.

8.7. Bateria em três domínios

A bateria mede o sistema em três acervos de áreas diferentes (segurança do trabalho, produto e consumidor, licitações), com perguntas escritas depois do desenvolvimento. O acervo de licitações repete as duas leis do anterior e acrescenta a Lei 10.520, de modo que os dois não são independentes. As perguntas têm cinco formas: *aberta* (pede um dado), *confirmação* (sim ou não), *existência* (o acervo trata do assunto?), *premissa falsa* (a pergunta traz um dado errado, e a resposta certa é corrigi-lo) e *negativa*. São 71 perguntas: 31 pares de literal e paráfrase (Seção 7) e 9 negativas sem par. Cada pergunta foi feita três vezes, num total de 213 respostas.

Com a pergunta original, a taxa de acerto foi de 69,5%, com os três acervos entre 62,3% e 82,6% e intervalos de confiança que se sobrepõem: de 43,5% a 81,2% em segurança do trabalho, de 65,2% a 95,7% em produto e consumidor e de 44,0% a 84,0% em licitações. O sistema acertou cerca de 7 em cada 10 respostas (intervalo de 95% de 59,2% a 79,3%) e, com 23 a 25 perguntas por acervo, não dá para dizer que um acervo foi mais fácil que outro. Por pergunta, 49 das 71 acertaram nas três repetições, 21 erraram nas três e só 1 oscilou: os erros se repetem entre execuções. A página certa esteve entre as fontes em 58,1% das respostas, abaixo do acerto, porque uma resposta certa pode vir de outra página com a mesma informação, o que é comum em acervos com documentos sobrepostos.

Esses números usam o gabarito revisado depois da primeira medição. Na versão original, em duas formas de pergunta bastava a resposta conter "há" ou "não" para contar

como certa, e uma recusa sempre igual ("Não há informações sobre isso nos documentos fornecidos") acertaria 24 das 71 perguntas sem responder nada. A revisão passou a exigir, na premissa falsa, o dado correto e deixou de aceitar recusas e respostas que dizem o contrário quando a pergunta tem resposta; todas as respostas que mudaram de certa para errada, ou o contrário, foram conferidas à mão. Com a revisão, o acerto caiu de 81,7% para 69,5% (Anexo A8).

Por forma da pergunta (tabela no repositório, Apêndice A), o sistema reconheceu a falta de informação em todas as 27 respostas negativas, e a premissa falsa foi a mais difícil: em só 13 de 36 respostas o sistema corrigiu o dado errado. Nas demais, disse que o acervo não trazia a informação ou respondeu dentro da premissa, como na pergunta sobre como abrir uma licitação na modalidade "tomada de preços", em que ensinou o procedimento da lei revogada. É a bajulação descrita por Sharma et al. [2024], somada ao problema de vigência da seção anterior.

Redação da pergunta. Por resposta, o acerto foi 88,2% nas literais e 41,9% nas paráfrases. Como as duas versões de um par pedem a mesma coisa, a comparação é feita par a par, e cada versão conta como certa quando acertou nas três repetições. Dos 31 pares, 17 tiveram o mesmo resultado nas duas versões, e 14 foram discordantes, todos a favor da literal: a literal acertou e a paráfrase errou, sem nenhum caso contrário. Se a redação não fizesse diferença, cada um desses 14 pares teria a mesma chance de pender para um lado ou para o outro, e a chance de os 14 caírem do mesmo lado por acaso é menor que 0,001 ($p < 0{,}001$ no teste de McNemar). Isso aconteceu nos três acervos: 7 pares em segurança do trabalho, 4 em produto e consumidor e 3 em licitações. Esse resultado depende da revisão do gabarito: com a versão original, eram 8 pares a favor da literal e 2 a favor da paráfrase ($p = 0{,}109$). O sistema acerta muito mais quando a pergunta usa as palavras do documento, o que confirma a fraqueza da busca por palavras descrita na Seção 3.1: quem não conhece o termo técnico tem menos chance de achar o trecho.

8.8. Primeira tentativa: enriquecer o índice

Se o usuário não usa as palavras do documento, uma saída é acrescentar ao índice as palavras que ele usaria. A primeira intervenção fez isso: para cada página, o LLM gerou um resumo e perguntas que a página responde, escritas como um leigo as escreveria. O pré-registro previa três braços (só o conteúdo, conteúdo mais resumo, conteúdo mais perguntas geradas), 16 pares de literal e paráfrase e os três acervos da bateria. A execução, porém, usou um só acervo, o de segurança do trabalho, com 12 perguntas: 5 pares de literal e paráfrase e 2 negativas. Também acrescentou um quarto braço, exploratório, que dá a cada página a maior entre duas notas: a do conteúdo e a das perguntas geradas. Pelo desvio em relação ao pré-registro, o resultado é exploratório.

Uma primeira execução foi descartada: as fontes eram idênticas nos quatro braços, porque a troca de braço não chegava ao processo que respondia; o experimento foi refeito com um processo da API por braço, e o pré-registro recebeu um adendo. Na reexecução, as fontes mudaram entre os braços em 8 das 12 perguntas, mas o acerto quase não mudou: 7 de 12 nos braços só com o conteúdo e com as perguntas geradas, e 8 de 12 no braço com o resumo, que é o padrão do sistema, e no exploratório. Com 5 paráfrases e uma repetição, o experimento não teria como detectar um efeito moderado. A explicação mais

provável é que o modelo, mesmo instruído a escrever como leigo, repete o vocabulário do documento.

8.9. Segunda tentativa: expandir a pergunta

A segunda intervenção vai no sentido contrário: em vez de levar as palavras do leigo ao documento, leva as palavras do documento à pergunta, como em Jagerman et al. [2023]. Espera-se que, diante de uma pergunta leiga, seja mais fácil o modelo chegar ao termo técnico (de "capacete e luva" a "EPI") do que prever, na indexação, todas as formas como um leigo perguntaria. A hipótese pré-registrada foi que a expansão melhoraria as paráfrases e deixaria as literais quase iguais.

O modelo recebe só o texto da pergunta e devolve termos técnicos relacionados, que são acrescentados à busca. Ele não vê o acervo nem o gabarito, e as 71 perguntas são as mesmas da bateria. O modelo que expande é o mesmo que depois responde. Para a pergunta "Quem paga o capacete e a luva que eu uso no serviço?", que errava nas três repetições, a expansão gerou *equipamento, proteção, individual, EPI, fornecedor, empregador, custo, pagamento*, e a resposta passou a citar a página certa da NR-6.

Com a expansão, o acerto das paráfrases subiu de 41,9% para 83,9% e ficou perto do das literais, que passou de 88,2% para 90,3% (Tabela 6). Nas mesmas 71 perguntas, 14 passaram a acertar nas três repetições e nenhuma deixou de acertar ($p < 0{,}001$ no teste de McNemar): de 49 para 63 das 71. O resultado não depende da revisão do gabarito: com o gabarito original, o efeito já era significativo (10 ganhos e 1 perda, $p = 0{,}012$; Anexo A8). A expansão agiu na busca: a página certa passou a estar entre as fontes em 82,3% das respostas, contra 58,1%. Os *tokens* enviados para responder cresceram 13,4%, e cada pergunta nova exige uma chamada a mais ao modelo.

Tabela 6. Expansão da pergunta nas 71 perguntas da bateria. As três primeiras linhas contam respostas (cada pergunta foi feita três vezes); a última conta perguntas. As 9 negativas, sem par, entram só nas linhas de todas as perguntas.

	Sem expansão	Com expansão
Literais, 31 perguntas (% de acerto)	88,2	90,3
Paráfrases, 31 perguntas (% de acerto)	41,9	83,9
Todas, 71 perguntas (% de acerto)	69,5	88,7
Perguntas certas nas três repetições	49 de 71	63 de 71

O controle é a execução anterior da bateria, feita no mesmo dia, sem sorteio entre as duas condições. Os três acervos são documentos públicos, que o modelo provavelmente viu no treinamento: "empregador", gerado para a pergunta sobre quem paga o equipamento, é ao mesmo tempo uma ponte de vocabulário e a própria resposta, e o ganho não pode ser estendido a documentos internos de uma organização. E não houve comparação com expansão sem LLM nem com busca híbrida. Como a primeira ocorrência de cada pergunta depende do modelo, a expansão ficou desligada por padrão; a partir da segunda, o resultado sai de um *cache* e a busca volta a ser determinística.

8.10. Edital municipal (exploratório)

Por fim, o sistema foi aplicado ao edital de um concurso público municipal, com 52 páginas, tabelas de cargos que atravessam páginas e dois capítulos com o mesmo número por erro do próprio edital. As 18 perguntas formam nove pares de literal e paráfrase, e cada par foi escrito para provocar um tipo de falha. O sistema acertou 15 de 18. As duas versões da pergunta que pede o total de vagas somando uma tabela de três páginas erraram, porque a busca trouxe só parte das páginas e o modelo somou o que viu. A terceira falha foi de busca: a pergunta pelo "Capítulo XVI" não trouxe a página do capítulo, porque o numeral romano pesa pouco no BM25. Somar linhas espalhadas por muitas páginas é um limite concreto de uma busca com número fixo de fontes e com teto de texto enviado ao modelo (Seção 6.5).

9. Discussão

Resposta à pergunta de pesquisa. A Introdução desdobrou a pergunta em três condições. A Tabela 7 resume o que os dados dizem sobre cada uma.

Tabela 7. Resposta à pergunta de pesquisa, condição por condição.

Condição	Resposta	Por quê
(a) Toda fonte mostrada tem origem lida do banco	Sim	O documento é ligado por chave estrangeira, e dois testes de integração conferem que ela existe nas seis tabelas de unidades e que o banco recusa um parágrafo sem documento. No acervo de ensino, a página certa, lida do banco, esteve entre as fontes em 195 de 201 respostas. Em C1, as 155 unidades têm origem completa no banco. A página e a região são gravadas na indexação; a página não é validada pelo banco, e um trecho que atravessa páginas aponta só a primeira.
(b) A busca sobre as unidades guardadas em tabelas, com a nota calculada na aplicação e sem vetores, acha o trecho certo em perguntas novas	Em parte	Acerto de 69,5% (cerca de 7 em 10). Com as palavras do documento, 88,2%; com outras palavras, 41,9%. A expansão por LLM leva a 88,7%, mas é experimental e foi medida só em documentos públicos. Não havia acerto mínimo fixado antes da medição; o "Em parte" é um julgamento sobre esses números.
(c) Conferir a fonte confirma que a resposta está certa	Não	A lei revogada é citada com a página certa: quem confere a fonte encontra o texto citado, sem aviso de que ele foi revogado. O sistema também não liga cada frase à fonte que a sustenta.

A resposta à pergunta de pesquisa é, portanto, afirmativa para a primeira condição e parcial para as outras duas, na escala medida (os três acervos da bateria têm de três a cinco documentos cada, e as perguntas são em português). Foi possível construir o sistema e garantir que cada fonte possa ser conferida usando só modelagem relacional, sem banco de vetores. Conferir a fonte, porém, não garante que a resposta esteja certa nem que a norma citada ainda esteja em vigor.

O que a garantia de origem cobre. Com o documento inteiro, o modelo citou a página certa tantas vezes quanto com a busca (189 de 201 respostas nos dois casos, aceitando também o número impresso na página). A diferença é que, com a busca, o leitor recebe uma página que vem do banco e não pode ter sido inventada pelo modelo. Essa garantia vem de guardar cada unidade junto com a sua origem. Se a busca fosse por vetores guardados numa coluna da mesma tabela, a garantia seria a mesma, porque quem a sustenta é a forma de guardar os dados. Como as duas buscas ficaram próximas em C1 e um índice de vetores bem configurado foi estável na escala do ThingIndex, dispensar o banco de vetores foi uma escolha de simplicidade. Os dados de C1, em que os dois tipos de busca erram por razões diferentes, indicam a busca híbrida dentro do próprio PostgreSQL como o próximo passo.

Limitações medidas. A redação teve o maior efeito: as paráfrases acertaram 41,9% contra 88,2% das literais, e, com a expansão, as paráfrases chegaram a 83,9%. A vigência foi o erro mais grave: o sistema trata como independentes documentos que se revogam e responde com fonte correta e conteúdo desatualizado. É um erro que conferir a fonte não revela, e por isso, na versão atual, o sistema não é adequado para acervos com normas revogadas, a menos que a coleção contenha só as versões vigentes. Também ficaram fora: respostas que exigem somar linhas de muitas páginas, limitadas pelo número de fontes, e coleções acima do limite de carga, cujas unidades excedentes, dos documentos enviados por último, saem da busca sem aviso ao usuário.

Sobre o método. O acervo de desenvolvimento chegou a 100,0%, resultado que não se repetiu em nenhum acervo de teste e que indica que o ajuste nele inflou o número. A primeira execução do enriquecimento deu um resultado nulo que parecia sólido, mas a troca de braço não chegava ao processo que respondia, e a medição foi refeita. A revisão do gabarito da bateria mostrou que recusas e até respostas opostas eram aceitas como acerto; com ela, o acerto caiu de 81,7% para 69,5% (Seção 8.7). As três correções foram feitas a partir dos dados brutos, guardados pergunta a pergunta, e estão registradas no Anexo A8.

9.1. Ameaças à validade

Amostras pequenas. C1 tem 16 consultas com resposta, pouco para qualquer comparação de busca ser significativa. A bateria tem 71 perguntas, e as duas versões de um par não são independentes. Os acervos de legislação e do edital têm uma repetição.

Um único anotador. O sistema foi ajustado no acervo de ensino. Todos os gabaritos foram escritos por uma só pessoa, sem um segundo anotador nem medida de concordância entre anotadores, e as paráfrases foram escritas por quem já tinha lido o documento. O critério por expressão regular pode aceitar resposta incompleta e recusar resposta certa redigida de outro modo; os dois casos foram encontrados e corrigidos onde vistos.

O que se mediu. A página na fonte e a página no texto não medem se o trecho sustenta a frase da resposta. A origem no índice foi medida num *corpus* sem paginação, e a região do parágrafo não foi avaliada em separado.

Generalização. Todos os documentos estão em português, e o maior acervo tem 10 PDFs. A busca densa só foi comparada no *corpus* de controle; nos acervos reais não houve um braço com vetores. Um único modelo redigiu as respostas; uma medição

exploratória com outro modelo, maior, só no braço F, deu acerto de 74,2% contra 100,0% do modelo principal nas 66 perguntas que o outro modelo respondeu, o que indica que os números dependem do modelo. Das 6 restantes, 5 eram negativas cujo texto, com cinco documentos, passou da janela do modelo, e 1 esgotou o tempo. Os acervos de teste são públicos, e não se mediu o comportamento com documentos internos. A utilidade para usuários reais não foi avaliada.

10. Conclusão

Este trabalho construiu o ThingIndex, um sistema que guarda a estrutura dos PDFs em tabelas relacionais e responde perguntas mostrando o documento e a página de cada fonte (e, nos parágrafos, a região), e o avaliou em sete acervos. Os principais resultados foram:

- A página mostrada ao usuário vem do banco, e não do texto do modelo, e o vínculo com o documento é garantido por chave estrangeira; as 155 unidades de C1 têm a origem gravada no banco.
- Nos acervos com perguntas escritas depois do desenvolvimento, a taxa de acerto foi de 69,5% com a pergunta original, cerca de 7 em cada 10 respostas. As paráfrases acertaram 41,9% e as literais, 88,2%, e a expansão da pergunta elevou o acerto a 88,7%, em configuração experimental, desligada por padrão e medida só em documentos públicos.
- Na mediana, buscar trechos usou 1/26 dos *tokens* de entregar o documento inteiro ao modelo, sem perda de acerto no acervo de desenvolvimento (72 de 72 perguntas contra 71 de 72).
- Busca por palavras e busca por significado ficaram próximas no *corpus* de controle (13 de 16 consultas contra 11 de 16), sem diferença que os testes estatísticos pudessem confirmar; dispensar o banco de vetores foi uma escolha de simplicidade de operação, sem ganho de qualidade demonstrado.
- O envio de documentos usa *outbox* transacional e fila com novas tentativas; o teste de caos derrubou a fila, o banco e o armazenamento e o sistema voltou sozinho, e 20 envios simultâneos foram todos indexados. Na revisão do código, descobriu-se que faltava no banco a chave estrangeira entre as unidades e o documento; ela foi criada, e hoje dois testes de integração conferem que ela existe e funciona.
- O erro mais grave foi o da norma revogada: o sistema cita, com a página certa, uma lei que já não vale, e conferir a fonte não revela o erro.

O modelo relacional sustenta a guarda das unidades e a origem de cada fonte: a auditabilidade vem desse esquema, que comportaria também vetores numa coluna das mesmas tabelas.

Para a vigência, o modelo relacional oferece um caminho: registrar o período de validade de cada artigo do documento, porque a revogação pode ser parcial, e comparar esses períodos em SQL. Os trabalhos futuros, em ordem de prioridade, são: tratar a vigência por unidade do documento; avaliar o sistema com usuários reais, medindo o tempo até a resposta e se as fontes são de fato conferidas; medir se o trecho exibido sustenta a frase da resposta; ampliar os gabaritos, com um segundo anotador e documentos que o modelo não conhece; e comparar a expansão e a busca híbrida no próprio PostgreSQL.

O código, os gabaritos, os resultados brutos, os *scripts* que geram e conferem os números deste artigo e os anexos estão no repositório do projeto, `https://github.com/EduardoMG12/ThingIndex`.

Declaração sobre o uso de inteligência artificial generativa

O ThingIndex foi projetado e desenvolvido pelo autor. Assistentes de IA generativa (ChatGPT, da OpenAI, e Claude, da Anthropic) foram usados como ferramentas de apoio à programação e à revisão do texto, incluindo ortografia e adequação ao limite de páginas. A pergunta de pesquisa, o desenho dos experimentos, as hipóteses, os gabaritos e as decisões técnicas são do autor, que revisou todo o conteúdo e assume responsabilidade integral por ele. Os números reportados vêm dos arquivos de resultado versionados e são conferidos por *scripts*.

Referências

Buneman, P., Khanna, S., e Tan, W.-C. (2001). Why and where: A characterization of data provenance. In Van den Bussche, J. e Vianu, V., editors, *Database Theory — ICDT 2001*, volume 1973 of *Lecture Notes in Computer Science*, pages 316–330, Berlin, Heidelberg. Springer.

Chase, H. (2022). LangChain. Repositório de software. Disponível em: `https://github.com/langchain-ai/langchain`. Acesso em: 27 set. 2026.

Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6):377–387.

Cormack, G. V., Clarke, C. L. A., e Buettcher, S. (2009). Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759. ACM.

Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Metropolitansky, D., Ness, R. O., e Larson, J. (2024). From local to global: A graph RAG approach to query-focused summarization. arXiv preprint arXiv:2404.16130.

Es, S., James, J., Espinosa Anke, L., e Schockaert, S. (2024). RAGAs: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, pages 150–158. Association for Computational Linguistics.

Furnas, G. W., Landauer, T. K., Gomez, L. M., e Dumais, S. T. (1987). The vocabulary problem in human-system communication. *Communications of the ACM*, 30(11):964–971.

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., e Wang, H. (2023). Retrieval-augmented generation for large language models: A survey. arXiv preprint arXiv:2312.10997.

Hevner, A. R., March, S. T., Park, J., e Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1):75–106.

Jagerman, R., Zhuang, H., Qin, Z., Wang, X., e Bendersky, M. (2023). Query expansion by prompting large language models. arXiv preprint arXiv:2305.03653.

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., e Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38.

Kane, A. (2026). pgvector: Open-source vector similarity search for Postgres. Repositório de software, versão 0.8.6. Disponível em: `https://github.com/pgvector/pgvector`. Acesso em: 27 set. 2026.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., e Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M. F., e Lin, H., editors, *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. Curran Associates, Inc.

Li, Z., Li, C., Zhang, M., Mei, Q., e Bendersky, M. (2024). Retrieval augmented generation or long-context LLMs? a comprehensive study and hybrid approach. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 881–893.

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., e Liang, P. (2024). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.

Manning, C. D., Raghavan, P., e Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press, New York.

McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157.

Peffers, K., Tuunanen, T., Rothenberger, M. A., e Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3):45–77.

Rashkin, H., Nikolaev, V., Lamm, M., Aroyo, L., Collins, M., Das, D., Petrov, S., Tomar, G. S., Turc, I., e Reitter, D. (2023). Measuring attribution in natural language generation models. *Computational Linguistics*, 49(4):777–840.

Richardson, C. (2018). *Microservices Patterns: With Examples in Java*. Manning Publications, Shelter Island.

Robertson, S. e Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389.

Sharma, M., Tong, M., Korbak, T., Duvenaud, D., Askell, A., Bowman, S. R., Cheng, N., Durmus, E., Hatfield-Dodds, Z., Johnston, S. R., Kravec, S., Maxwell, T., McCandlish, S., Ndousse, K., Rausch, O., Schiefer, N., Yan, D., Zhang, M., e Perez, E. (2024). Towards understanding sycophancy in language models. In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024)*.

Thakur, N., Reimers, N., Rücklé, A., Srivastava, A., e Gurevych, I. (2021). BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In Vanschoren, J. e Yeung, S., editors, *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, volume 1.

VectifyAI (2025). PageIndex: Document index for vectorless, reasoning-based RAG. Repositório de código aberto (licença MIT). Disponível em: `https://github.com/VectifyAI/PageIndex`. Acesso em: 5 set. 2026.

Wang, L., Yang, N., Huang, X., Yang, L., Majumder, R., e Wei, F. (2024). Multilingual E5 text embeddings: A technical report. arXiv preprint arXiv:2402.05672.

Xu, R., Qi, Z., Guo, Z., Wang, C., Wang, H., Zhang, Y., e Xu, W. (2024). Knowledge conflicts for LLMs: A survey. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 8541–8565. Association for Computational Linguistics.

Apêndice A: Anexos no repositório

Conforme orientação do colegiado, os anexos estão em PDF na pasta `anexos/` do repositório do projeto, `https://github.com/EduardoMG12/ThingIndex`, com um índice em `anexos/README.md`:

- A1 – requisitos funcionais e não funcionais;
- A2 – diagrama de casos de uso;
- A3 – diagrama de sequência do envio de um documento e de uma pergunta;
- A4 – recorte do esquema com o caminho da proveniência;
- A5 – fluxo da recuperação;
- A6 – telas do protótipo;
- A7 – resultados do teste de busca (C1) explicados, com a tabela completa de medidas, os intervalos de confiança e os índices aproximados do pgvector; a lista das 90 comparações pareadas está em `docs/article/tables`;
- A8 – registro das alterações feitas depois de medir;
- A9 – esquema completo do banco, com as 25 tabelas;
- A10 – uma pasta por acervo, com os PDFs indexados, exceto o do edital municipal, que não está versionado, e um arquivo com as perguntas daquele acervo, a resposta esperada e a página-alvo;
- Tabelas de apoio dos resultados, em `docs/article/tables/apoio-resultados.tex`: acerto e página no texto por tipo de pergunta no acervo de ensino, com médias e faixa de *tokens*; legislação por tipo de pergunta; bateria por acervo e por forma da pergunta; C1 por tipo de consulta; detalhes da expansão.

Os dados brutos, os pré-registros e os *scripts* de conferência estão nas pastas indicadas no índice.

Apêndice B: Pontuação da recuperação

A nota de cada unidade parte do BM25 ($k_1 = 1{,}2$; $b = 0{,}75$), calculado separadamente para cada tipo de unidade da coleção, multiplicado por 100 e arredondado. A nota é então multiplicada por um peso fixo por tipo: 3 para seções, 2 para páginas e trechos e 5 para linhas de tabela. As seções recebem ainda um acréscimo proporcional à confiança da extração e um acréscimo fixo quando não são de primeiro nível. Uma linha de tabela precisa conter pelo menos dois termos significativos da pergunta (ou o único, se a pergunta tiver um só), para que um número solto não traga uma tabela inteira, e linhas com menos da metade da nota da melhor são descartadas. Os pesos e limites foram escolhidos durante o desenvolvimento, pelo efeito nas perguntas do acervo de ensino, e não foram testados um a um, retirando cada peso para medir seu efeito; o código está em `api/internal/usecase/collection_retrieval.go`.